



UNIVERSIDAD ADOLFO IBÁÑEZ

Simulaciones Basadas en Agentes



MCP

Model Context Protocol



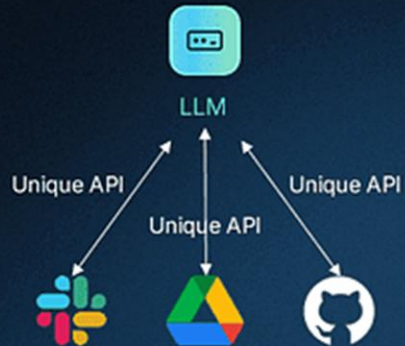
Protocolo de Contexto de Modelo (MCP)

¿Alguna vez has deseado que tu asistente de IA pudiera hacer más que solo conversar? ¿Qué pasaría si realmente pudiera realizar tareas o buscar datos en tiempo real por ti? Ahí es donde entra el Protocolo de Contexto de Modelo (MCP). MCP es un estándar abierto creado por Anthropic para ayudar a los modelos de IA —especialmente a los grandes modelos de lenguaje (LLM) como Claude— a conectarse con herramientas y fuentes de datos externas de una manera simple y estandarizada. Para un principiante en IA, puedes pensar en MCP como un “conector universal” que permite que un modelo de IA se comuniquen con diferentes sistemas —como APIs, bases de datos o incluso una calculadora— sin necesidad de escribir código personalizado para cada uno. Esto hace que la IA sea más poderosa al permitirle usar herramientas e información del mundo real.

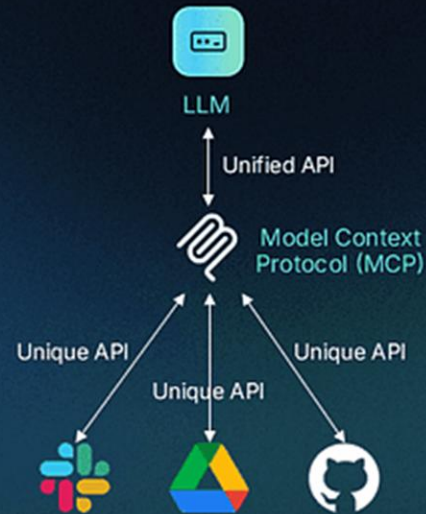
Protocolo de Contexto de Modelo (MCP)

descope

Before MCP



After MCP



Protocolo de Contexto de Modelo (MCP)

¿Qué es MCP?

MCP es un protocolo (un conjunto de reglas) que define cómo las aplicaciones de IA pueden comunicarse con herramientas externas o fuentes de datos. Utiliza una configuración de cliente-servidor:

Servidores MCP: Son programas que proporcionan herramientas o datos específicos, como una API del clima, un lector de archivos o, en nuestro caso, una calculadora. Estos servidores “hablan” el lenguaje MCP, así que cualquier IA puede entenderlos.

Clientes MCP: Son las aplicaciones de IA que se conectan a los servidores MCP para usar esas herramientas. El cliente puede pedirle datos al servidor o indicarle que realice una acción.

Protocolo de Contexto de Modelo (MCP)

Aquí tienes una analogía sencilla: Imagina que MCP es como un traductor útil en una biblioteca. Tú (el cliente de IA) quieres información de diferentes libros (herramientas), pero los libros están escritos en varios idiomas. El traductor (MCP) conoce todos los idiomas y puede conseguirte la información, así que no necesitas aprender cada idioma por tu cuenta.

La gran ventaja de MCP es que está estandarizado: una vez que una herramienta está configurada como un servidor MCP, cualquier cliente MCP (como un asistente de IA) puede usarla sin trabajo adicional.

¿Cómo funciona MCP?

MCP funciona creando un puente entre la IA y las herramientas externas:

1. **Disponibilidad de herramientas:** Un servidor MCP ofrece herramientas específicas (por ejemplo, una función de calculadora) y describe lo que hacen.
2. **Conexión:** El cliente MCP se conecta al servidor y descubre qué herramientas están disponibles.
3. **Uso de herramientas:** El cliente puede usar estas herramientas cuando las necesita, ya sea directamente o dejando que un modelo de IA (como Claude) decida cuándo utilizarlas.
4. **Comunicación bidireccional:** La IA puede tanto obtener datos (por ejemplo, el resultado de un cálculo) como ejecutar acciones (por ejemplo, enviar un comando), lo que la hace muy interactiva.

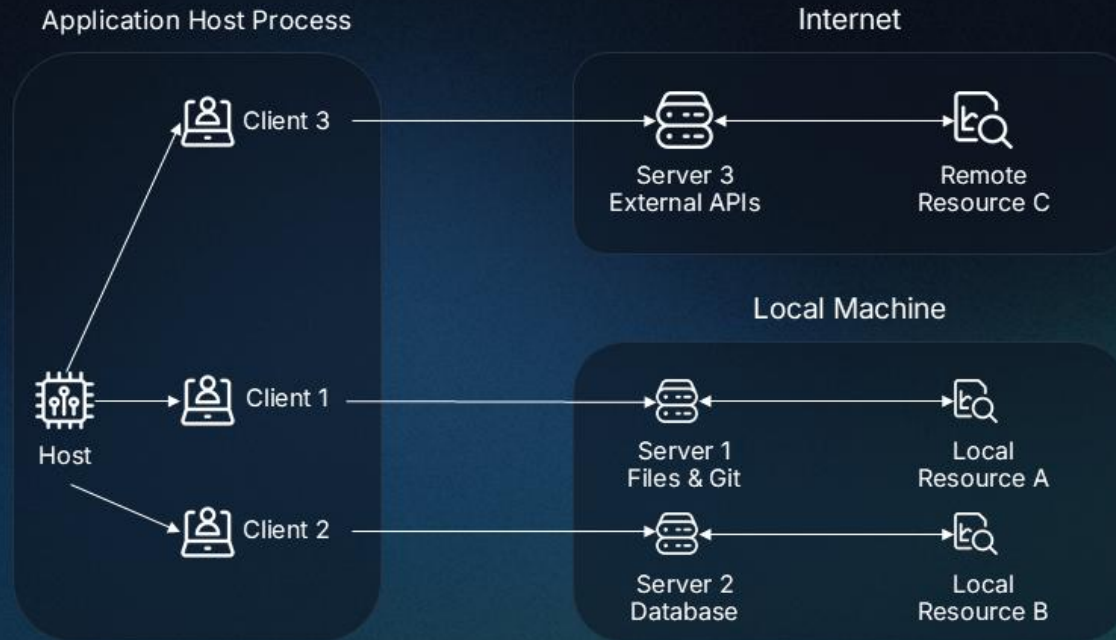
Protocolo de Contexto de Modelo (MCP)

Por ejemplo, si le preguntas a una IA: "¿Cuánto es $5 + 3$?", un cliente MCP podría usar un servidor MCP con una herramienta de calculadora para obtener la respuesta (8) y devolvérsela a la IA para que te responda. Cuando se combina con un modelo de IA como Claude, el cliente le envía al modelo una lista de herramientas disponibles. El modelo entonces decide si alguna herramienta puede ayudar a responder tu pregunta y le indica al cliente que la use.

Protocolo de Contexto de Modelo (MCP)

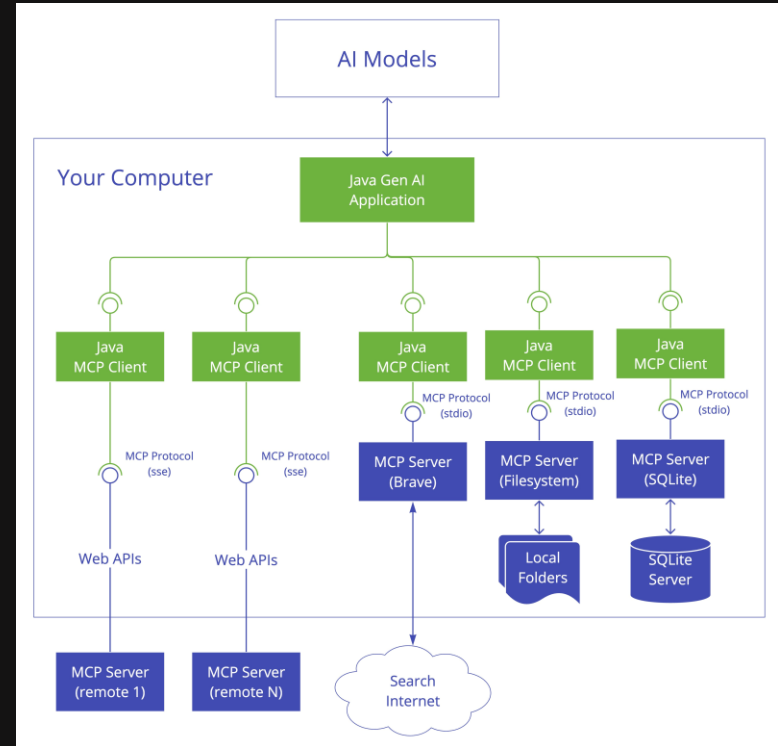
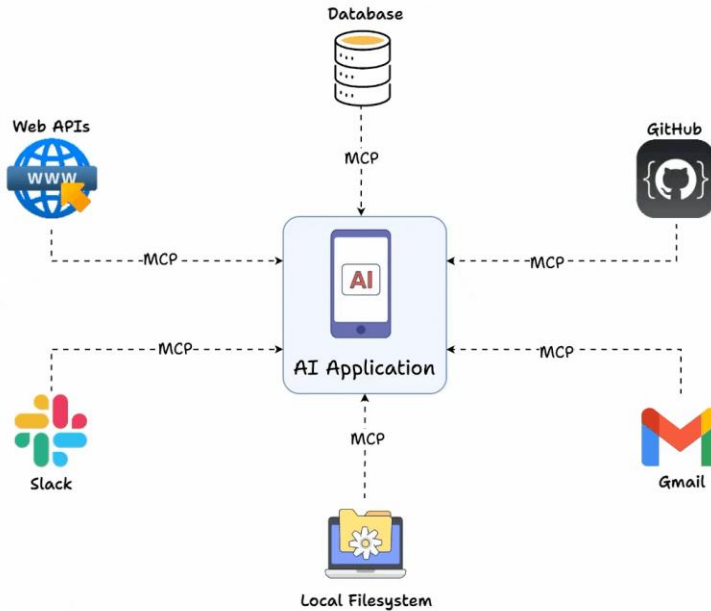
MCP Core Components

descope



Protocolo de Contexto de Modelo (MCP)

What is MCP ?



Protocolo de Contexto de Modelo (MCP)

Configurando el proyecto

Vamos a crear un pequeño proyecto para ver MCP en acción. Construiremos un asistente de IA que pueda responder preguntas de matemáticas (por ejemplo, "¿Cuánto es 5 por 7?") conectándose a una herramienta de calculadora a través de MCP. El proyecto tiene dos partes:

Parte 1: Construir un servidor y un cliente MCP básicos para mostrar cómo se comunican entre sí.

Parte 2: Añadir un modelo de IA (Claude) para que el asistente sea lo suficientemente inteligente como para manejar preguntas de los usuarios.

Protocolo de Contexto de Modelo (MCP)

ANTHROPIC

[Claude](#) [API](#) [Solutions](#) [Research](#) [Commitments](#) [Learn](#) [News](#)

[Try Claude](#)

Build with Claude

Create user-facing experiences, new products, and new ways to work with the most advanced AI models on the market.

[Start building](#)

[Developer docs](#)



<https://console.anthropic.com/dashboard>

Protocolo de Contexto de Modelo (MCP)

Paso 1: Configura tu entorno

Primero, vamos a crear un espacio de trabajo e instalar las herramientas necesarias.

Crea un entorno virtual (esto mantiene tu proyecto separado de otras cosas de Python):

```
bash
```

[Copy](#)[Edit](#)

```
python -m venv mcp-env
```

```
py -3.10 -m venv MCP
```

```
MCP\Scripts\activate
```

```
pip install mcp anthropic
```

Protocolo de Contexto de Modelo (MCP)

Paso 2: Crear el servidor MCP

El servidor ofrecerá una herramienta: `evaluate_expression`, que toma una expresión matemática (como "2 + 3") y devuelve el resultado.

Crea un archivo llamado `calculator_server.py`.

Protocolo de Contexto de Modelo (MCP)

Explicación del código

Servidor: Creamos un servidor MCP usando `FastMCP()`.

Herramientas: Definimos una herramienta llamada `evaluate_expression` usando el decorador `@server.tool`. Esta herramienta recibe una cadena de texto (por ejemplo, "2 + 3") y utiliza la función `eval` de Python para calcular el resultado.

Ejecución: `server.run()` inicia el servidor, haciendo que la herramienta esté disponible en `localhost:8080` (una dirección local en tu computadora).

Advertencia: Usar `eval` puede ser riesgoso con entradas no confiables, ya que ejecuta cualquier código de Python. Para esta demostración simple está bien, ya que lo usamos localmente, pero en proyectos reales deberías usar un método más seguro (como un parser matemático).

Protocolo de Contexto de Modelo (MCP)

Paso 3: Crear el cliente MCP (básico)

Ahora vamos a crear un cliente básico que se conecte al servidor y use directamente la herramienta de calculadora.

Crea un archivo llamado `client_basic.py`.

Protocolo de Contexto de Modelo (MCP)

Explicación del código

Imports: Usamos `ClientSession` y `StdioServerParameters` de `mcp`, y `stdio_client` para manejar la comunicación por stdio, tal como se muestra en la documentación.

Parámetros del servidor:

- `command="python"` : Ejecuta Python como el programa principal.
- `args=["calculator_server.py"]` : Especifica el script del servidor que se va a ejecutar.
- `env=None` : Usa las variables de entorno por defecto.

Función asíncrona `run` :

- `stdio_client(server_params)` : Inicia el servidor como un subprocesso y proporciona funciones de lectura y escritura para la comunicación por stdio.
- `ClientSession(read, write)` : Establece la sesión usando estas funciones.
- `await session.initialize()` : Establece la conexión.
- `await session.list_tools()` : Lista las herramientas disponibles (útil para depuración).
- `await session.call_tool()` : Llama a la herramienta `evaluate_expression` con el input del usuario.

Interacción con el usuario:

El programa solicita al usuario una expresión matemática y muestra el resultado en pantalla.

Protocolo de Contexto de Modelo (MCP)

Paso 4: Ejecutar el servidor y el cliente

En este paso, solo ejecutaremos el cliente, y este se encargará de iniciar el servidor automáticamente. Por supuesto, también puedes separarlos y ejecutar el servidor de forma independiente si lo prefieres.

Abre una terminal, activa el entorno virtual (`source mcp-env/bin/activate`), y ejecuta:

```
bash
```

[Copy](#)[Edit](#)

```
python client_basic.py
```

De esta manera, el cliente llamará al servidor por ti.

Protocolo de Contexto de Modelo (MCP)

Processing request of type ListToolsRequest

=== Available Tools ===

```
- ('meta', None)
- ('nextCursor', None)
- ('tools', [Tool(name='evaluate_expression', description='Evaluates a mathemat
=====
```

Enter a math expression (e.g., '5 * 7'):

Enter a math expression (e.g., '5 * 7'): 5*8

Processing request of type CallToolRequest

=== Calculation Result ===

Expression: 5*8

Result: meta=None content=[TextContent(type='text', text='40', annotations=None)]

=====

Paso 5: Integrar el modelo de IA

Ahora, vamos a hacer que el cliente sea más inteligente agregando a Claude. La IA se encargará de interpretar las preguntas del usuario y usará la herramienta de calculadora cuando sea necesario (por ejemplo, para preguntas como "¿Cuánto es 5 por 7?").

Crea un archivo llamado `client_ai.py`.

Cómo funciona

Configuración del servidor y cliente:

- El servidor de calculadora (`calculator_server.py`) se inicia como un subprocesso.
- El cliente MCP se conecta a él e inicializa la sesión.

Descubrimiento de herramientas:

- El cliente lista las herramientas disponibles (por ejemplo, `evaluate_expression`) y las imprime para confirmar que están ahí.
- Las herramientas se formatean en un esquema que Claude puede entender, asumiendo que cada objeto de herramienta tiene nombre, descripción y un `input_schema` .
- Puedes hacer que las herramientas sean dinámicas según las disponibles en el servidor MCP, pero en este ejemplo están escritas a mano.

Interacción con el usuario:

- Se le pide al usuario que haga una pregunta (por ejemplo, "¿Cuánto es 5 por 7?" o "Háblame de matemáticas").
- Esto es más flexible que `client_basic.py`, que solo aceptaba expresiones matemáticas.

Procesamiento de la IA:

- La consulta se envía a Claude junto con la definición de herramientas.
- Claude decide si:
 - Responde directamente (por ejemplo, para preguntas no matemáticas).
 - Usa la herramienta `evaluate_expression` (por ejemplo, para $5 * 7$).

Ejecución de la herramienta:

- Si Claude solicita la herramienta, el cliente la llama con los argumentos proporcionados (por ejemplo, `{"expression": "5 * 7"}`).
- El resultado (por ejemplo, "35") se envía de vuelta a Claude en un mensaje de seguimiento.

Salida final:

- La respuesta de Claude se imprime con separadores decorativos, manteniendo el estilo de `client_basic.py` .

¡Recuerda!

Antes de ejecutar el código, reemplaza `your_api_key_here` con tu clave API real de Anthropic.

Protocolo de Contexto de Modelo (MCP)

Paso 6: Probar el asistente de IA

1. Ejecuta el cliente de IA con el siguiente comando:

```
bash
```

[Copy](#)[Edit](#)

```
python client_ai.py
```

2. Escribe una pregunta como:

“¿Cuánto es 5 por 7?”

y observa la respuesta del asistente.

```
Ask me a question (e.g., 'What's 5 times 7?'): What's 5 times 7?
```

```
=== Assistant's Response ===  
The result of 5 times 7 is 35.  
=====
```

Protocolo de Contexto de Modelo (MCP)

Entendiendo el papel de MCP

Esto es lo que hemos aprendido hasta ahora:

- **Los servidores MCP** proporcionan herramientas (como nuestra calculadora) de una forma que cualquier cliente MCP puede usar fácilmente.
- **Los clientes MCP** se conectan a estos servidores y pueden usar las herramientas directamente (Parte 1) o dejar que un modelo de IA decida cuándo usarlas (Parte 2).
- Cuando se combina con una IA como Claude, MCP permite que la IA aproveche capacidades externas, haciéndola más inteligente y útil.

<https://github.com/modelcontextprotocol/python-sdk>

```
py -3.10 -m venv MCP
MCP\Scripts\activate
```



UNIVERSIDAD ADOLFO IBÁÑEZ

Gracias!

ANEXO



ANTHROPIC



MEMORY
HUMAN MIND





UAI
UNIVERSIDAD ADOLFO IBÁÑEZ

Simulaciones Basadas en Agentes

